

Cross-Rate Domain Separation in Feed-Forward Sponge Constructions

Tsinghua Hash Lab

Abstract. Sponge constructions and their feed-forward variants can define families of hash functions by varying parameters over a shared permutation. We study domain separation within such families: whether instances with different parameters behave as independent random functions. For capacity feed-forward, we give a simple three-block message construction that yields predictable relations between the outputs of instances with different rates under stated conditions. The construction applies even with multirate padding to specified parameter families allowed by the published **Sponge-F** and **SPONGE-DM^c** definitions. We analyze its cost and success probability and explain why the same construction does not apply to several other sponge variants.

Keywords: Sponge construction · Domain separation · Feed-forward

1 Introduction

The sponge construction turns a fixed-width permutation into a variable-input, variable-output function. Its b -bit state is partitioned into an r -bit rate and a $c = b - r$ bit capacity. Absorption XORs each padded message block into the rate before applying the permutation; squeezing reads rate bits and applies further permutations as needed [1, Section 2.2].

Sponge variants modify different aspects of this design. Duplex interleaves input and output calls while retaining the state, supporting applications such as authenticated encryption [2]. Feed-forward variants instead alter the update: all or part of a permutation input is XORed into its output. **Sponge-F** [3] and **SPONGE-DM^c** [6] both feed forward the capacity part, reading output from the end and the beginning of the state, respectively. Other variants include **SPONGE-DM**, which uses full-state feed-forward, and **SPONGE-EDM^a**, which feeds forward the last a input bits before an outer permutation [6]. These changes need not preserve every property of the original sponge family.

Domain separation is one such property. If parameter choices sharing a primitive serve as separate domains, the relevant ideal object is a family of independent random oracles. Single-instance collision or preimage bounds do not establish that joint behavior. Bertoni et al. [1, Sections 3.1.1 and 6.5, Theorem 10] discuss disjoint input encodings and prove multirate domain separation for ordinary sponge with $10*1$ padding. For **Sponge-F**, Guo et al. [3, Section 1.3, pp. 74–75] identify an interaction between the initialization vector (IV) and the final-block marker θ when $IV_2 = IV_1 \oplus \theta$, and recommend avoiding it. We study how changing the rate affects domain separation in capacity-feed-forward sponge families.

2 Technical Overview

We study domain separation between parameter instances of the same hash family: whether an adversary querying multiple instances and their shared public permutation can efficiently distinguish the family from a collection of independent random functions. We focus on the capacity feed-forward update used by Sponge-F and SPONGE-DM^c, comparing different-rate instances of either construction that share a permutation and use multirate padding. Our observation is that *a change in the capacity feed-forward boundary creates a difference that the next message block can cancel*. The search chooses a first block for which the compensating final block satisfies the padding rule. Figure 1 shows the whole mechanism.

Consider two instances using the same public b -bit permutation p . Their absorption rates satisfy $0 < r < r' < b$, and their capacities are $c = b - r$ and $c' = b - r'$, respectively. The two rate boundaries partition each state into three segments of lengths $(r, r' - r, c')$.

For the rate- r instance, let B_i be the i th padded message block, S_i the state after absorbing it, and X_i the permutation input. The corresponding values in the rate- r' instance are B'_i , S'_i , and X'_i . At rate r ,

$$\begin{aligned} X_i &= S_{i-1} \oplus (B_i \parallel 0^c), & S_i &= F_r(X_i), \\ F_r(X) &= p(X) \oplus (0^r \parallel X[r : b]). \end{aligned} \quad (1)$$

Here F_r is the update after message injection; $X[u : v]$ selects bits $u, \dots, v - 1$, numbered from zero in absorption order. We use $\oplus$ for bitwise XOR, $\parallel$ for concatenation, and 0^m for m zero bits. The second instance uses the same rule with r', c' . Both start from a fixed common state I whose first r' bits are zero. A fixed common mask K , also zero in its first r' bits, may be used for the final block. In that block, set $X_i = S_{i-1} \oplus (B_i \parallel 0^c) \oplus K$ before applying F_r ; $I = K = 0^b$ is allowed. The key identity is

$$F_r(X) \oplus F_{r'}(X) = 0^r \parallel X[r : r'] \parallel 0^{c'}. \quad (2)$$

The middle segment is capacity at rate r but message-controlled at rate r' . Ordinary sponge has no such rate-dependent update difference.

3 A three-block construction

Use 10^*1 padding, denoted by pad_r at rate r . For a bit-oriented interface, a message ending two bits before a block boundary receives the suffix $\tau = 11$, so set $k = 2$. For byte-oriented messages and byte-aligned rates, use the whole padding byte $\tau = 10000001$ and set $k = 8$. Here $k = |\tau|$ is the number of padding bits; assume $k \leq \min(r, r' - r)$. All bit strings below are written from left to right in absorption order.

Try distinct r -bit strings A and compute

$$S_1 = p(I \oplus (A \parallel 0^c)) \oplus I, \quad V = S_1[r : r'],$$

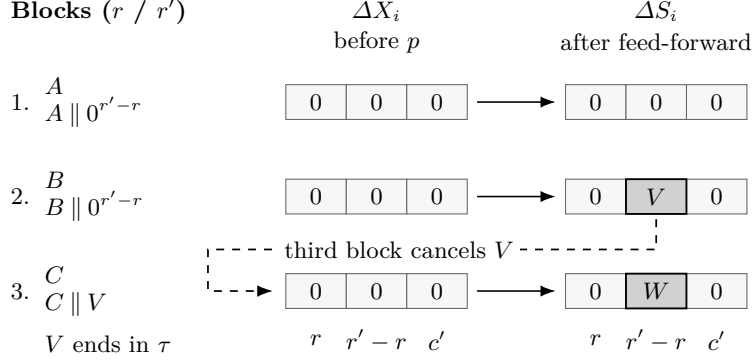


Fig. 1. Three updates, with $\Delta X_i = X_i \oplus X'_i$ and $\Delta S_i = S_i \oplus S'_i$ denoting the XOR differences between the instances. Each block pair lists the rate- r block above the rate- r' block. All three permutation inputs match ($\Delta X_i = 0$). The second feed-forward creates V in the middle segment; the next message cancels it. The final feed-forward may leave $W = X_3[r : r'] \neq 0$, but the first r and last c' state bits match. Segment widths are schematic.

until the last k bits of V equal τ . Choose any r -bit B and any r -bit C ending in τ . The two padded messages are

$$\begin{aligned} \text{pad}_r(M) &= A \parallel B \parallel C, \\ \text{pad}_{r'}(M') &= (A \parallel 0^{r'-r}) \parallel (B \parallel 0^{r'-r}) \parallel (C \parallel V). \end{aligned} \quad (3)$$

Removing the last k bits of each string gives a valid message: the removed bits are exactly its padding. Thus the search constrains only a short suffix of V , not its entire $r' - r$ -bit value.

Theorem 1. *Every successful construction (3) gives equal first r and last c' bits of the final absorption states. Let $T \leq 2^r$ be the budget of distinct first-block trials, and p_T the probability of finding a valid pair within this budget. For a uniform random p ,*

$$p_T \geq 1 - (1 - 2^{-k})^T \geq 1 - e^{-T2^{-k}}. \quad (4)$$

The first-block search uses at most T permutation queries and $O(b)$ bits of working memory.

Proof. Equal first inputs have a zero middle segment, giving common S_1 . Equal second inputs have middle segment V , so (2) gives $S_2 \oplus S'_2 = 0^r \parallel V \parallel 0^{c'}$. The third block pair cancels this difference; the common mask preserves equal permutation inputs. By (2), only the middle segment may differ after the final update. Distinct trials sample permutation outputs without replacement. A 2^{-k} fraction meet the padding condition, and failures cannot reduce the next hit probability, proving (4). One counter and a constant number of states suffice.

Taking $T = 2^k$ gives success probability at least $1 - e^{-1}$: four trials for the bit interface, or 256 for the byte interface. These are first-block search queries; evaluating the two three-block hashes adds only constant work when the needed output lies in the first output segment. The search cost does not grow with c' .

Output relations. The final states need not be identical. Let ℓ and ℓ' be the numbers of bits the two instances actually return before any further state update, limited by the requested digest lengths. If these bits are read from the left end of the state, the common digest prefix has length

$$t = \min\{r, \ell, \ell'\}. \quad (5)$$

If each segment is read as a state suffix, the two first output segments have a common suffix of length $t = \min\{c', \ell, \ell'\}$. This is a digest-suffix relation when both digests are returned entirely in these segments. For example, direct suffix outputs of 1024 and 512 bits at $(b, r, r') = (1600, 512, 1024)$ have the same last 512 bits. The same reasoning applies to other aligned output positions within the common state segments.

The output relation can be assessed by a direct probability comparison. After at most q queries in total to two independent random functions, the probability that any pair of outputs from different functions matches at the specified t positions is at most $q^2 2^{-t}$, by a union bound. Here q is the ideal-function query budget, including the permutation simulator's queries; T instead counts first-block trials. Simulator queries matter because the messages depend on permutation answers. When this bound is much smaller than the success probability in Theorem 1, the matching output segment distinguishes the real family from independent random functions. This comparison concerns separate domains; prefix consistency within a single XOF is part of its definition.

Other padding. Under the same assumptions on I and K , 10^* padding needs no search: every message M with $|M| < r$ gives the same embedded padded input at both rates, with a zero middle segment, hence the same final state. The three-block construction also extends to fixed removable suffixes: the smaller-rate suffix must fit in C , and the larger-rate suffix must fit in V , so their lengths are at most r and $r' - r$, respectively. Appending each suffix to an arbitrary prefix of the required length must give a valid encoded message, and removing it must respect the message interface. Under these conditions, different short domain suffixes alone do not prevent compensation; the search cost is set by the larger-rate suffix length. Length-encoding padding requires a separate analysis.

4 Scope and Applications

We identify when the three-block construction applies and where its alignment fails.

4.1 Applicability to Sponge Variants

Sponge-F and SPONGE-DM^c share the capacity feed-forward update, while their initialization, final-block processing, and output rules must be checked separately.

Capacity feed-forward: Sponge-F. The construction applies to the following parameter family in the published definition [3, Section 4.1 and Figure 3, pp. 83–85]. Choose each instance’s capacity-sized IV as all zeros and its marker θ as all zeros except for the last bit, giving common $I = 0^b$ and $K = 0^{b-1} \parallel 1$. Set each instance’s minimum output length and squeezing rate equal to its requested digest length, no greater than its capacity. This makes the output a direct state suffix. With 10^*1 padding and the same rate-gap condition, the digest suffixes match.

Capacity feed-forward: SPONGE-DM^c. The construction applies. Sun et al.’s Algorithm 1 and Section 3.1 (p. 140), together with Section 4.2 (p. 144), specify (1) with $I = K = 0^b$ [6]. Keep the permutation fixed, choose rates satisfying $k \leq \min(r, r' - r)$, and use the padding and exposed output bits described above. The resulting digest prefixes match.

Ordinary sponge. The construction does not apply with 10^*1 padding. Equal permutation inputs give equal next states at both rates, so no feed-forward difference is available for compensation. On this aligned path, the final padding 1 occupies different state positions at the two rates; zero-extension cannot make both final blocks valid and equal. The classical multirate theorem establishes joint security in the ideal-permutation model [1, Section 6.5, Theorem 10].

Full-state feed-forward: SPONGE-DM. The update $p(X) \oplus X$ is independent of the rate. With zero initialization and 10^*1 padding, simulate both instances by a base instance at rate r' with 10^* padding. Zero-extend each padded block to r' bits and remove the final 10^* padding. The encoded lengths modulo r' are $r - 1$ and $r' - 1$, giving disjoint domains [1, Section 6.5, Theorem 10]. Squeezing uses the same permutation: set the base squeezing rate to the larger one and recover each target output from the appropriate block prefixes. Combining this encoding with the single-instance indistinguishability theorem in the random-permutation model gives a joint bound [6, Section 5.2, Theorem 1, p. 147]. Use the base capacity $c' = b - r'$, squeezing parameters, and total permutation-call count, including output expansion.

Encrypted feed-forward: SPONGE-EDM^a. The published definition feeds forward the last a state bits; its security proof uses the same random permutation for the inner and outer absorption calls and for squeezing [6, Section 4.3, pp. 145–146]. For fixed a , the absorption update is independent of the rate, and the preceding reduction applies under the same initialization, padding, and output conditions. If the two rates instead use $a = c$ and $a = c'$, the outer permutation can move the

difference into the capacity shared by both instances, where the next message cannot modify it. Equation (2) no longer holds. The direct compensation step is therefore not guaranteed; joint security in this case requires a separate analysis.

Initialization and other output choices. Different initial states or finalization rules may prevent alignment. Ascon-Hash256 and Ascon-XOF128 both use 10^* padding but different 64-bit IVs. Replacing these IVs by a common value would make the former’s 256-bit digest a prefix of the latter’s output for every message [5, Algorithms 2, 5–6]. Separating fixed digest lengths at one absorption rate requires an additional encoding or initialization distinction.

4.2 Applications to Candidate Hash Functions

China’s NGCC program, launched in 2025, provides concrete applications through its publicly available hash submissions [4]. ZC-DMC (H033) and Iphe (H043) admit the three-block construction, using the short-suffix extension for ZC-DMC. Across their respective 512- and 1024-bit variants, this gives a common 64-bit digest prefix for ZC-DMC and a common 512-bit digest suffix for Iphe. Mozi (H005) and Eijen (H021) exhibit the simpler one-block relation with 10^* padding: on the same message shorter than 960 bits, Mozi-384 is a prefix of Mozi-512, and Eijen-512 is a suffix of Eijen-768 and Eijen-1024. These examples support including domain separation between parameter instances in candidate evaluations alongside single-instance collision and preimage security.

Concrete ZC-DMC construction (1536-bit state). The construction can be instantiated directly with the ZC-DMC witness supplied with this work. Use one 1536-bit permutation P , the all-zero initial state, and the 512-bit and 768-bit digest instances. Since the specification sets $c = n + 64$, their parameters are

$$(r, c) = (960, 576) \quad \text{and} \quad (r', c') = (704, 832).$$

Both instances use the padding $M \parallel 01 \parallel 10^*1$. Choose the removable short suffix $\tau = 0111$ in the bit interface; the search then constrains only the last four bits of the compensating middle difference. First choose a block for which the two feed-forward updates create a compensable 576-bit capacity difference, choose a second block so that this difference lies in the 704-bit instance’s message part, and use a third block to cancel it while completing the final padding. The reference witness gives messages of 2876 and 2108 bits, respectively, with the final 0111 removed to obtain the unpadded messages. Their first 64-bit squeeze block is the same:

7609c584e8a27ced.

This is a concrete three-block instantiation on one permutation. After the final absorption, the first 704 state bits agree, so the first 64-bit output block specified by ZC-DMC agrees as well. Further squeezing invokes additional permutation calls; the claim here concerns this first block. The witness was computed from the specified capacity feed-forward, initialization, padding, and squeeze boundaries, so it parameterizes the generic construction with actual algorithm values rather than relying only on a length-nesting argument.

Concrete Iphe construction (2048-bit state). The accompanying H043 reproduction package gives a full-round witness for Iphe-512 and Iphe-1024. The two instances share a 2048-bit permutation and the all-zero initial state. Iphe-512 has rate 1472 bits (23 64-bit words), while Iphe-1024 has rate 960 bits (15 words), so the rate gap is 512 bits; their capacities are 576 and 1088 bits, respectively. Absorb the same two 960-bit blocks in both instances, appending 512 zero bits to each block in Iphe-512. The first two permutation inputs are therefore identical. After the second feed-forward, the 512-bit difference in words 15–22 lies in the Iphe-1024 capacity but in the extra Iphe-512 rate area.

The third block cancels this difference: Iphe-512 absorbs it in its extra 512 rate bits, making the complete third permutation input identical. In the concrete witness, the Iphe-1024 final block contains 958 message bits followed by its two padding bits. Iphe-512 uses the same first 960 bits, appends 510 correction bits from the state difference, and receives the final two 1 padding bits from the interface. The search therefore needs only the last two difference bits to equal 11, requiring about $2^2 = 4$ trials on average. Deterministic seed 3 gives a 4414-bit Iphe-512 message and a 2878-bit Iphe-1024 message. The reference implementations satisfy

$$\text{Iphe}_{512}(M_{512}) = \text{suffix}_{512}(\text{Iphe}_{1024}(M_{1024})),$$

with common 512-bit value

```
53cee235555dd69149d78d37c00039f0
baa83a6beb4af42e15e7122dde584b07
5c5a79b151e2052f0a03fe7a3c625848
f6bbe7b538f1a47b022ba8373aca8e37
```

. The complete messages and digests are stored in `attack_vectors.json` in the H043 reproduction package; the claim targets the submitted reference implementations and their API.

References

1. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Cryptographic sponge functions. Tech. Rep. Version 0.1, Keccak Team (2011), <https://keccak.team/files/CSF-0.1.pdf>
2. Bertoni, G., Daemen, J., Peeters, M., Van Assche, G.: Duplexing the sponge: Single-pass authenticated encryption and other applications. In: SAC 2011. LNCS, vol. 7118, pp. 320–337. Springer (2012). doi:10.1007/978-3-642-28496-0_19
3. Guo, C., Hu, K., Jiang, S., Fan, Y., Fu, Y., Preneel, B., Wang, M.: Permutation-based hash from non-idealized assumptions: Adding feed-forward to sponge. In: CRYPTO 2026. LNCS, vol. 16805, pp. 65–97. Springer (2026). doi:10.1007/978-3-032-35412-9_3
4. Institute of Commercial Cryptography Standards: Next-generation commercial cryptographic algorithms program (NGCC) (2026), <https://www.niccs.org.cn/>, official website and Round 1 hash submissions; accessed 22 September 2026

5. Sönmez Turan, M., McKay, K.A., Chang, D., Kang, J., Kelsey, J.: Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions. Tech. Rep. SP 800-232, NIST (2025). doi:10.6028/NIST.SP.800-232
6. Sun, S., Li, S., Zhang, Z., Lefevre, C., Mennink, B., Qin, Z., Feng, D.: Permutation-based hashing with stronger (second) preimage resistance. In: CRYPTO 2026. LNCS, vol. 16805, pp. 131–162. Springer (2026). doi:10.1007/978-3-032-35412-9_5